EXPERIMENTS 29 TO 32

29.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    int item;

    for (item = 1; item <= 5; item++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer produced: %d\n", item);
        in = (in + 1) % SIZE;
```

```
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg) {
    int item, i;

    for (i = 1; i <= 5; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }

    return NULL;
}

int main() {
```

```
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

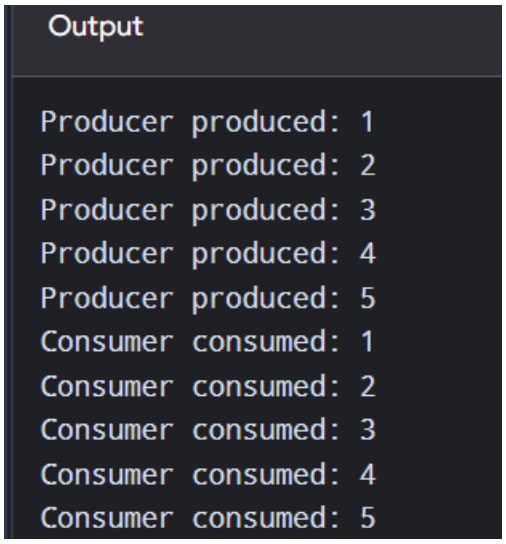
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

OUTPUT:



30.

```
#include <stdio.h>
#include <pthread.h>

pthread_t thread1, thread2;

void *function1(void *arg) {
    printf("Thread 1 is running.\n");
    pthread_exit(NULL);
}

void *function2(void *arg) {
    printf("Thread 2 is running.\n");
    return NULL;
}
```

```
int main() {
    if (pthread_create(&thread1, NULL, function1, NULL) != 0) {
        printf("Error creating Thread 1.\n");
        return 1;
    }

    if (pthread_create(&thread2, NULL, function2, NULL) != 0) {
        printf("Error creating Thread 2.\n");
        pthread_join(thread1, NULL);
        return 1;
    }

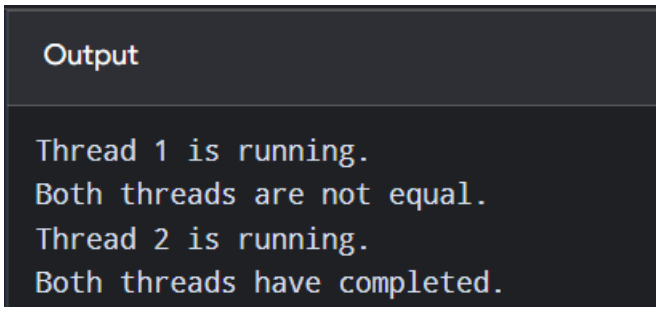
    if (pthread_equal(thread1, thread2))
        printf("Both threads are equal.\n");
    else
        printf("Both threads are not equal.\n");

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Both threads have completed.\n");

    return 0;
}
```

OUTPUT:



31.

```
#include <stdio.h>

int main() {
    int pages[50], frames[10];
    int n, f, i, j;
    int faults = 0, hits = 0;
    int next = 0, found;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
```

```
scanf("%d", &f);

if (n <= 0 || n > 50 || f <= 0 || f > 10) {
    printf("Invalid input.\n");
    return 1;
}

for (i = 0; i < f; i++) {
    frames[i] = -1;
}

printf("\nPage\tFrames\t\tStatus\n");

for (i = 0; i < n; i++) {
    found = 0;

    for (j = 0; j < f; j++) {
        if (frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }

    if (found) {
        hits++;
    } else {
```

```
            frames[next] = pages[i];
            next = (next + 1) % f;
            faults++;
        }

        printf("%d\t", pages[i]);

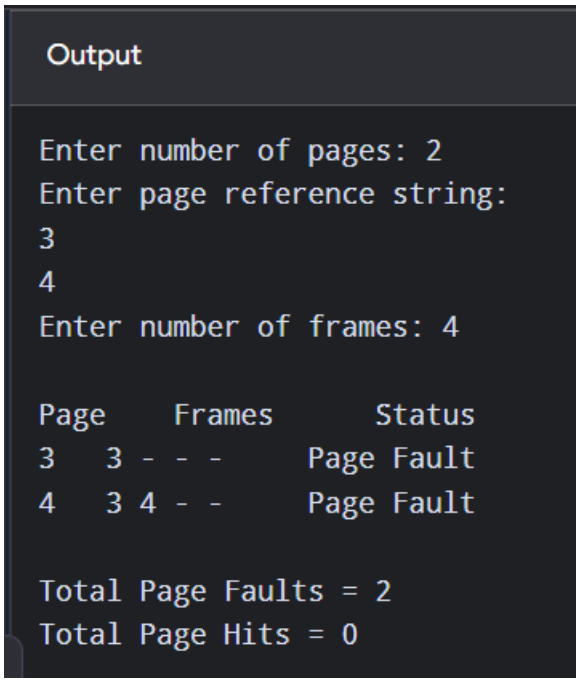
        for (j = 0; j < f; j++) {
            if (frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }

        if (found)
            printf("\tHit\n");
        else
            printf("\tPage Fault\n");
    }

    printf("\nTotal Page Faults = %d\n", faults);
    printf("Total Page Hits = %d\n", hits);

    return 0;
}
```

OUTPUT:



32.

```
#include <stdio.h>

int main() {
    int pages[50], frames[10], lastUsed[10];
    int n, f, i, j, k;
    int faults = 0, hits = 0;
    int found, lru;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    if (n <= 0 || n > 50) {
```

```
    printf("Invalid number of pages.\n");
    return 1;
}

printf("Enter page reference string:\n");
for (i = 0; i < n; i++) {
    scanf("%d", &pages[i]);
}

printf("Enter number of frames: ");
scanf("%d", &f);

if (f <= 0 || f > 10) {
    printf("Invalid number of frames.\n");
    return 1;
}

for (i = 0; i < f; i++) {
    frames[i] = -1;
    lastUsed[i] = -1;
}

printf("\nPage\tFrames\t\tStatus\n");

for (i = 0; i < n; i++) {
    found = 0;
```

```
for (j = 0; j < f; j++) {
    if (frames[j] == pages[i]) {
        found = 1;
        lastUsed[j] = i;
        break;
    }
}

if (found) {
    hits++;
} else {
    lru = -1;

    for (j = 0; j < f; j++) {
        if (frames[j] == -1) {
            lru = j;
            break;
        }

        if (lru == -1 ||
            lastUsed[j] < lastUsed[lru]) {
            lru = j;
        }
    }
```

```
            frames[lru] = pages[i];
            lastUsed[lru] = i;
            faults++;
        }

        printf("%d\t", pages[i]);

        for (k = 0; k < f; k++) {
            if (frames[k] == -1)
                printf("- ");
            else
                printf("%d ", frames[k]);
        }

        printf("\t%s\n", found ? "Hit" : "Page Fault");
    }

    printf("\nTotal Page Faults = %d\n", faults);
    printf("Total Page Hits = %d\n", hits);

    return 0;
}
```

OUTPUT:

Output

```
Enter number of pages: 3
Enter page reference string:
4
5
6
Enter number of frames: 7

Page    Frames      Status
4   4 - - - - - -   Page Fault
5   4 5 - - - - -   Page Fault
6   4 5 6 - - - -   Page Fault

Total Page Faults = 3
Total Page Hits = 0
```